

CS201 Lab 5

Data Structures and Algorithms

Shreya Agarwal

Rachit Nimavat

Spring 2026

This lab builds directly upon your `IntVector` library from Labs 2, 3, and 4. You will need your `vector.h`, `vector.c`, and `Makefile`. If your implementation of `IntVector` was buggy, you may start fresh by downloading a copy from the [CS201 labs Github](#).

Part 1: Stock Price Rolling Peaks

You are given daily stock prices of a company in an array. You are analyzing the volatility of a stock over time. To identify local peaks, you need to track the maximum stock price within a “moving window” of fixed size k as it slides forward one day at a time. Given an array of n daily stock prices and an integer $k \leq n$ (representing the window size), return an array where each element is the maximum price observed in the corresponding window of k days. Write a $O(nk)$ solution first, and then optimize it to $O(n)$ time using a well-known¹ data structure.

Example: Input: [100, 80, 50, 60, 70, 40, 90] and $k=3$. Output: [100, 80, 70, 70, 90].

Part 2: Doubly Linked List based IntVector

Our `IntVector` supports `append()`, `search()` and `pop_index()` operations with a singly linked list as a backend. However, `pop_index()` is not a natural fit for singly linked lists since we need to traverse the list to find the node just before the one we want to remove. Implement a doubly linked list version of `IntVector` and replace `pop_index()` with `pop_node()` that takes a pointer to the node to be removed. This allows for $O(1)$ removal of nodes when we have a direct pointer to them. Update your `IntVector` library and modify your code for part 2 of lab-4 so it still solves the Josephus problem.

Questions to Ponder Over this Week

1. What is the space complexity of your deque implementation for Part 1 in terms of the input size n and window size k ?
2. This is probably the first time you are implementing a doubly linked list and would have gotten a few SegFaults along the way. What are some common mistakes you made while implementing the doubly linked list?
3. When the logic becomes more complex, it is easy to forget to update all the pointers correctly. What strategies did you use to debug your doubly linked list implementation? Once the code become even more complex, it makes sense to use

¹Use a (monotonic) Double-ended Queue (Deque). While you can implement a deque using your `IntVector`, it is recommended to implement it using the two-pointers approach.

structured debugging tools like `gdb` to step through your code and inspect the state of your linked list at various points in time. You can find a tutorial on using `gdb` [here](#).

4. In the Array version, accessing the k -th element of `IntVector` took $O(1)$ time. Does this change the overall time complexity of your Josephus solution from Lab 3 or Lab 4? If so, from what to what?
5. In the Array version, storing 1000 integers took roughly $1000 * 4 = 4000$ bytes. In the Singly Linked List version, we needed a Node struct for every integer. Each Node contains an integer (4 bytes) and a pointer to the next Node (8 bytes on a 64-bit system). So, each Node takes $4 + 8 = 12$ bytes and for 1000 integers, you need $1000 * 12 = 12000$ bytes. For the Doubly Linked List version, each Node contains an integer (4 bytes), a pointer to the next Node (8 bytes), and a pointer to the previous Node (8 bytes). So, each Node takes $4 + 8 + 8 = 20$ bytes and for 1000 integers, you need $1000 * 20 = 20000$ bytes. This is a significant memory overhead compared to the array version. Is the flexibility of the doubly linked list worth this penalty? What do you gain from using a doubly linked list over a singly linked list? Can you think of scenarios where one is preferred over the other?